

SESSION – 15

Sort Characters By Frequency

```
public class Solution {
    public String frequencySort(String s) {
        Map<Character, Integer> map = new HashMap<>();
        for (char c : s.toCharArray())
            map.put(c, map.getOrDefault(c, 0) + 1);

        List<Character> [] bucket = new List[s.length() + 1];
        for (char key : map.keySet()) {
            int frequency = map.get(key);
            if (bucket[frequency] == null) bucket[frequency] = new ArrayList<>();
            bucket[frequency].add(key);
        }

        StringBuilder sb = new StringBuilder();
        for (int pos = bucket.length - 1; pos >= 0; pos--)
            if (bucket[pos] != null)
                for (char c : bucket[pos])
                    for (int i = 0; i < pos; i++)
                        sb.append(c);

        return sb.toString();
    }
}
```

The screenshot displays a coding platform interface for the problem "451. Sort Characters By Frequency". The problem description states: "Given a string *s*, sort it in **decreasing order** based on the **frequency** of the characters. The **frequency** of a character is the number of times it appears in the string. Return the sorted string. If there are multiple answers, return any of them."

Example 1:
Input: *s* = "tree"
Output: "eert"
Explanation: 'e' appears twice while 'r' and 't' both appear once. So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

Example 2:
Input: *s* = "cccaaa"
Output: "aaaccc"

The "Testcase" section shows three cases, all marked as "Accepted". Case 1 has an input of "tree" and an expected output of "eert".

The "Code" section shows the following Java code:

```
1 public class Solution {
2     public String frequencySort(String s) {
3         Map<Character, Integer> map = new HashMap<>();
4         for (char c : s.toCharArray())
5             map.put(c, map.getOrDefault(c, 0) + 1);
6
7         List<Character> [] bucket = new List[s.length() + 1];
8         for (char key : map.keySet()) {
9             int frequency = map.get(key);
10            if (bucket[frequency] == null) bucket[frequency] = new ArrayList<>();
11            bucket[frequency].add(key);
12        }
13
14        StringBuilder sb = new StringBuilder();
15        for (int pos = bucket.length - 1; pos >= 0; pos--)
16            if (bucket[pos] != null)
17                for (char c : bucket[pos])
18                    for (int i = 0; i < pos; i++)
19                        sb.append(c);
20
21        return sb.toString();
22    }
23 }
```